

# Bitmasking and Combinatorics

In competitive programming, you often encounter problems with small constraints, like  $N \leq 20$ . Whenever you see  $N$  hovering around 15 to 22, your brain should immediately scream: Bitmasking.

Here is how you use integers to represent and manipulate subsets.

## 1. The Core Concept: What is a Bitmask?

A bitmask is just an integer, but instead of caring about its decimal value, we care about its binary representation.

Imagine you have an array of items: ['Apple', 'Banana', 'Cherry']. You can represent any combination of these items using a 3-bit number.

- The 0-th bit represents 'Apple'.
- The 1-st bit represents 'Banana'.
- The 2-nd bit represents 'Cherry'.

If the bit is 1, the item is in the subset. If the bit is 0, the item is out.

- 000 (Decimal 0): The empty set [].
- 101 (Decimal 5): The set ['Apple', 'Cherry'].
- 111 (Decimal 7): The full set ['Apple', 'Banana', 'Cherry'].

## 2. Generating All Subsets in $O(2^N)$

Because a number with  $N$  bits can represent  $2^N$  different states (from 0 to  $2^N - 1$ ), counting up through normal integers literally generates every possible combination of your array.

Here is the standard boilerplate to process all subsets of an array of size  $N$ :

```
int n = 3;
// Total subsets = 2^n. We use 1 << n to quickly calculate this.
int total_subsets = 1 << n;

for (int mask = 0; mask < total_subsets; mask++) {
    // We are now looking at one specific subset (represented by 'mask')

    // Let's decode the mask to see which items are in this subset
    for (int i = 0; i < n; i++) {
        if ((mask & (1 << i)) != 0) {
            // The i-th bit is 1, so the i-th item is in this subset!
            // Process the item here...
        }
    }
}
```

This entirely replaces the need to write complex, messy recursive backtracking functions just to find combinations.

## 3. Set Theory Operations via $O(1)$ Bitwise Logic

Once your sets are represented as integers, standard set operations become single CPU instructions. Let  $A$  and  $B$  be two bitmasks representing two different subsets.

- Intersection ( $A \cap B$ ):  $A \& B$   
(Items present in BOTH sets. AND only keeps bits that are 1 in both).
- Union ( $A \cup B$ ):  $A | B$   
(Items present in AT LEAST ONE set. OR keeps a bit if it's 1 in either).
- Symmetric Difference ( $A \Delta B$ ):  $A \wedge B$   
(Items present in exactly one set, but not both. XOR acts as a toggle).
- Set Difference ( $A \setminus B$ ):  $A \& \sim B$   
(Items in  $A$  but strictly NOT in  $B$ . We invert  $B$  to destroy matching bits).
- Subset Check ( $A \subseteq B$ ):  $(A \& B) == A$  OR  $(A | B) == B$   
(If  $A$  is a subset of  $B$ , intersecting them should just yield  $A$ ).

## 4. Submask Enumeration in $O(3^N)$

Generating all subsets of an array is easy. But what if you are given a specific mask (let's say 10110), and you need to generate all the valid submasks of that specific mask? (Meaning, you can only turn 1s into 0s, but you can never turn a 0 into a 1).

A naive approach would be to loop from 0 to mask and check if (submask & mask) == submask. But if you do this inside a loop that is already iterating over all  $2^N$  masks, your time complexity blows up to  $O(4^N)$ , which will Time Out.

Here is the highly optimized  $O(3^N)$  loop that skips invalid states entirely:

```
int mask = 22; // Binary 10110

// We start with the submask equal to the mask itself
for (int submask = mask; submask > 0; submask = (submask - 1) & mask) {
    // Process the valid submask here
}
// Note: If you also need to process the empty set (0),
// you can do it outside the loop, as this loop terminates when submask reaches 0.
```

Why (submask - 1) & mask works:

1. Subtracting 1 from a number flips its rightmost 1 to a 0, and turns all trailing 0s into 1s.
2. However, this subtraction might introduce new 1s in positions where the original mask had 0s! (Which is illegal for a submask).
3. By immediately applying & mask, we forcefully clear out any of those illegal 1s, guaranteeing we drop down to the very next valid submask in descending order.

This specific loop is the beating heart of advanced techniques like Bitmask DP and SOS DP (Sum Over Subsets).